

Assignment No :- 4

Q.1. Consider the following code draw CFG and apply live variable analysis Explain. why constraint rules defined for assignment are sound.

input (x);

while (x > 1) { { y = x/2 ;

if (y > 3) .

{

x = x - y;

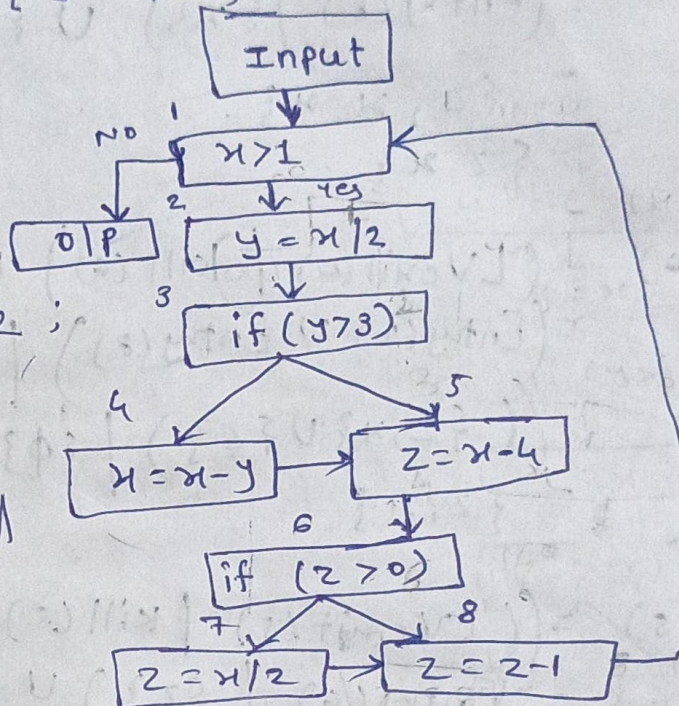
z = x - 4;

if (z > 0)

x = x/2 ;

z = z - 1;

Output (x);



→ Direction = Backward
meet opⁿ = U

Transfer function .

$$LV_{entry}(s) = (LV_{exit}(s) / kill(s)) \cup gen(s)$$

gen(s) = Variables on RHS of expression.

kill(s) = Variables on LHS of expression.

Statement	gen(s)	kill(s)	Entry(s)	Exit(s)
1	{x}	{φ}	{x}	Entry(2) ∪ φ
2	{x}	{y}	{x}	Entry(3)
3	{y}	{φ}	{x, y}	Entry(4) ∪ Entry(5)
4	{x, y}	{x}	{x, y}	Entry(5)
5	{x}	{z}	{x}	Entry(6)
6	{z}	{φ}	{z, x}	Entry(7) ∪ Entry(8)
7	{x}	{x}	{z, x}	Entry(8)
8	{z}	{z}	{z}	Entry(1)

$$LVEntry(8) = (LVexit(8) / kill(8)) \cup gen(8)$$

initial ϕ

$$= (\phi / \{z\}) \cup \{z\}$$

$$= \{z\}$$

$$LVEntry(7) = (LVexit(7) / kill(7)) \cup gen(7)$$

$$= (entry(8) / \{x\}) \cup \{x\}$$

$$= \{z, x, x\}$$

$$= \{z, x\}$$

$$LVEntry(6) = (LVexit(6) / kill(6)) \cup gen(6)$$

$$= (Entry(7) \cup Entry(8) / \{\phi\}) \cup \{z\}$$

$$= (\{z, x\} \cup \{z\} / \{\phi\}) \cup \{z\}$$

$$= \{z, x\}$$

$$LVEntry(5) = (LVexit(5) / kill(5)) \cup gen(5)$$

$$= Entry(6) / \{z\} \cup \{x\}$$

$$= \{z, x\} / \{z\} \cup \{x\}$$

$$= \{x\}$$

$$LVEntry(4) = (LVexit(4) / kill(4)) \cup gen(4)$$

$$= (\{x\} / \{x\}) \cup \{x, y\}$$

$$= \{x, y\}$$

$$LVEntry(3) = (LVexit(3) / kill(3)) \cup gen(3)$$

$$= (\{x, y\} / \{\phi\}) \cup \{y\} \cup (Entry(4) \cup Entry(5) / \{\phi\}) \cup \{y\}$$

$$= \{x, y\}$$

$$= (\{x, y\} \cup \{x\} / \{\phi\}) \cup \{y\}$$

$$LVEntry(2) = LVexit(2) / kill(2) \cup gen(2)$$

$$= (\{x, y\} / \{y\}) \cup \{x\}$$

$$= \{x\}$$

$$LVEntry(1) = (LVexit(2) \cup \{\phi\} / kill(1)) \cup gen(1)$$

$$= (\{x\} \cup \{\phi\} / \{\phi\}) \cup \{x\}$$

$$= \{x\}$$

In Second Iteration, we put the value

$$LVexit(8) = Entry(1) \Rightarrow \{x\}$$

$$\begin{aligned} LVEntry(8) &= (LVexit(8) \mid kill(8)) \cup gen(8) \\ &= (\{x\} \mid \{z\}) \cup \{z\} \\ &= \{x, z\} \end{aligned}$$

$$\begin{aligned} LVEntry(7) &= (LVentry(7) \mid kill(7)) \cup gen(7) \\ &= (LVentry(8) \mid \{x\}) \cup \{x\} \\ &= (\{x, z\} \mid \{x\}) \cup \{x\} \\ &= \{x, z\} \end{aligned}$$

$$\begin{aligned} LVEntry(6) &= (LVentry(6) \mid kill(6)) \cup gen(6) \\ &= (LVentry(7) \cup LVentry(8) \mid \{\phi\}) \cup \{z\} \\ &= (\{x, z\} \cup \{x, z\} \mid \{\phi\}) \cup \{z\} \\ &= \{x, z\} \end{aligned}$$

$$\begin{aligned} LVEntry(5) &= (LVentry(5) \mid kill(5)) \cup gen(5) \\ &= (LVentry(6) \mid \{z\}) \cup \{x\} \\ &= (\{x, z\} \mid \{z\}) \cup \{x\} \\ &= \{x\} \end{aligned}$$

$$\begin{aligned} LVEntry(4) &= (LVexit(4) \mid kill(4)) \cup gen(4) \\ &= (LVentry(5) \mid \{x\}) \cup \{x, y\} \\ &= (\{x\} \mid \{x\}) \cup \{x, y\} \\ &= \{x, y\} \end{aligned}$$

$$\begin{aligned} LVEntry(3) &= (LVexit(3) \mid kill(3)) \cup gen(3) \\ &= (LVentry(4) \cup Entry(5) \mid \{\phi\}) \cup \{y\} \\ &= ((\{x, y\} \cup \{x\}) \mid \{\phi\}) \cup \{y\} \\ &= \{x, y\} \end{aligned}$$

$$\begin{aligned} LVEntry(2) &= (LVexit(2) \mid kill(2)) \cup gen(2) \\ &= (LVentry(3) \mid kill \{y\}) \cup \{x\} \\ &= (\{x, y\} \mid \{y\}) \cup \{x\} \\ &= \{x\} \end{aligned}$$

$$\begin{aligned}
 LVEntry(1) &= (LVExit(1) \mid Kill(1)) \cup gen(1) \\
 &= (LVEntry(2) \cup \phi \mid \{ \phi \}) \cup \{x\} \\
 &= (\{x\} \mid \{ \phi \}) \cup \{x\} \\
 &= \{x\}
 \end{aligned}$$

Statements	gen	Kill	Entry	Exit
1	$\{x\}$	ϕ	$\{x\}$	$Entry(2) \cup \phi = \{x\}$
2	$\{x\}$	y	$\{x\}$	$Entry(3) = \{x, y\}$
3	$\{y\}$	ϕ	$\{x, y\}$	$Entry(4) \cup Entry(5) = \{x, y\}$
4	$\{x, y\}$	x	$\{x, y\}$	$Entry(5) = \{x\}$
5	$\{x\}$	z	$\{x\}$	$Entry(6) = \{x, z\}$
6	$\{z\}$	ϕ	$\{x, z\}$	$Entry(7) \cup Entry(8) = \{x, z\}$
7	$\{x\}$	x	$\{x, z\}$	$Entry(8) = \{x, z\}$
8	$\{z\}$	z	$\{x\}$	$Entry(1) = \{x\}$

Why constraint rules defined for assignment are sound.

It removes variables whose old values are overwritten

It keeps variables required for computation It ensures no needed value is lost.

Q.2. Prove the following Code using axiomatic Semantics.

$\{x = a \wedge y = b\}$

$t := x;$

$x := y;$

$y := t;$

$\{x = b \wedge y = a\}$

→ Precondition $\rightarrow \{x = a \wedge y = b\}$

assignment $\rightarrow t := x;$

$x := y;$

$y := t;$

Postcondition $\rightarrow \{x = b \wedge y = a\}$

$y := t \Rightarrow t = a \wedge x = b$

$x := y \Rightarrow y = b \wedge t = a$

$t := x \Rightarrow y = b \wedge x = a$

$wp(x = a \wedge y = b) == \{x = a \wedge y = b\}$

Why constraint rules defined for assignment a

Q.8.

Q.3 Apply Symbolic execution on the following code.

```
x := 3;  
y := x + 2;  
if (y > 4) then  
    z := y * 2  
else  
    z := y - 1;
```

→

```
x := 3;  
y := 3 + 2;
```

if (5 > 4) then → This condition is always True.
execute if statement.
z := 5 * 2

⇒ z = 10. ∴ x = 3, y = 5, z = 10.

Q.4. Solve the following

- what is fixed point Theory in DFA?

→ A fixed point of a function is a value that does not change when the function is applied.

- In DFA, we compute information (like live variable, reaching definitions, etc.) repeatedly until the result stops changing.

This stable result is called a fixed point.

eg. loop goes on changing the values after final stable state values not change. That is fixed point.

(i) explain with example, how operational Semantics evaluates the Conditional branching?

→ Operational Semantics explains how a program executes step by step. for Conditional branching it shows how the program chooses which branch to execute based on the condition.

eg. if (B) Then

```
    S1;  
else  
    S2;
```

∴ B is the boolean expression. if B is True Then S1 is executed. else S2 is executed if B is False.

In operational semantics, This is return as .

- $\langle \text{Statement}, \text{state} \rangle \rightarrow \langle \text{new statement} / \text{state} \rangle$

ex

$$\langle S_1, S \rangle \rightarrow S' \text{ --- Btt}$$

$$\langle S_2, S \rangle \rightarrow S' \text{ --- Bff}$$

$$\langle \text{if } (B) S_1 \text{ else } S_2, S \rangle \rightarrow S'$$

iii) Prove using operational semantic rule.

$$x := 1;$$

$$x := x + 2;$$

① Structural operational semantics.

$$\frac{[[x \mapsto 1]]_{S_0} \rightarrow [[x \mapsto 1]]_{S_1}}{\langle x = 1, S_0 \rangle \rightarrow S_1}$$

$$\frac{[[x \mapsto 1]]_{S_1} \rightarrow [[x \mapsto 1+2 \mapsto 3]]_{S'}}{\langle x := 1; x := x+2, S_0 \rangle \Rightarrow \langle x := x+2, S_1 \rangle \rightarrow S'}$$

$$\langle x := 1; x := x+2, S_0 \rangle \Rightarrow \langle x := x+2, S_1 \rangle \rightarrow S'$$

$$\frac{\frac{[[x \mapsto 1]]_{S_0} \rightarrow [[x \mapsto 1]]_{S_1} \quad \frac{[[x \mapsto 3]]_{S_1}}{\langle x \mapsto 1+2, S_1 \rangle \rightarrow S'}}{\langle x := 1, S_0 \rangle \rightarrow S_1 \quad \langle x := x+2, S_1 \rangle \rightarrow S'}{\langle x := 1; x := x+2; S_0 \rangle \rightarrow S'}$$

- Prove the following code using checklist of axiomatic semantics.

$$\{x = n\}$$

$$\text{while } x > 0 \text{ do } x := x - 1;$$

$$\{x = 0\}$$

$$\rightarrow \{P\} S \{C\}$$

$$P \Rightarrow x = n$$

$$C \Rightarrow x = 0$$

$$S \Rightarrow \text{while } x > 0 \text{ do } x = x + 1;$$

$$\text{invariance} \Rightarrow x \geq 0$$

Invariant is true in starting

$(x = n)$, applying body rule $x = x + 1$;

$x = n + 1$;

∴ This never reaches to 0

The invariant remains true.

But the loop goes infinite.

∴ The postcondition does not reach.

$$\{x = n\} \{x = x + 1\} \{x = n + 1\}$$

$$\{x = n\} \{x = x + 1\} \{x = n + 1\}$$

$$\{x = n\} \{x = x + 1\} \{x = n + 1\}$$

Prove the following code using Hoare's logic

$\{x = 0\}$

while $x > 0$ do

$x = x - 1$

end while

$x = 0$

$\{x = 0\}$

∴ while loop is correct

Invariant $x \geq 0$